Day 4 – RAG (Retrieval Augmented Generation) Deep Dive

Day 4 builds on embeddings and vectors (Day 3) and explains how we use them to build real-world AI systems using RAG. RAG allows an LLM to first retrieve relevant information from your own data and then generate answers based on that data. This turns the LLM from a guessing machine into a reading + answering machine.

1. What Problem RAG Solves

LLMs do not know your private data, can hallucinate, and have a limited context window. You cannot paste all your documents into the prompt. RAG solves this by fetching only the relevant pieces of data at query time and giving them to the LLM to answer from.

2. The RAG Pipeline (Step by Step)

- Ingest: Take documents (PDFs, text, code, DB rows), split them into chunks, and create embeddings for each chunk.
- Store: Save each chunk along with its embedding in a vector database.
- Retrieve: When a user asks a question, convert the question into an embedding and search for the top-k most similar chunks.
- Generate: Send the retrieved chunks along with the user question to the LLM and generate a final answer.

3. Example: Company FAQ Bot

You have company PDFs like HR policies and leave rules. During ingestion, these PDFs are split into chunks and stored as embeddings. When a user asks, “How many casual leaves do I get?”, the system retrieves the chunk that says “Employees are entitled to 12 casual leaves per year” and gives it to the LLM. The LLM then answers using this exact information. The answer is grounded in your data, not a guess.

4. Example: Codebase Assistant

Your FastAPI project code is stored as chunks with embeddings. When you ask, “Where is JWT validation implemented?”, the system retrieves the relevant code chunks (for example, from middleware.py or auth.py) and the LLM explains where and how JWT validation works. The LLM is reading your code at query time via retrieval.

5. Why Chunking is Important

Large documents contain many topics. If you store a whole document as one embedding, meanings get mixed and retrieval becomes poor. So documents are split into smaller chunks (for example, 300–1000 tokens) often with some overlap. This makes retrieval more precise and ensures important information near boundaries is not lost.

6. Vector Database and Top-K Search

A vector database stores text chunks along with their embedding vectors. When you search, you give it the question’s vector, and it returns the top-k most similar vectors (for example, top 3 or top 5). If k is too small, you may miss context. If k is too large, you may add noise and waste the

context window.

7. What the LLM Actually Sees

In a RAG system, the LLM receives the user question plus the retrieved chunks as context. The prompt usually instructs the LLM to answer using only this provided context. This keeps answers grounded and reduces hallucinations.

8. Common Failure Modes

- Bad chunking: chunks are too big or too small, causing poor retrieval.
- Bad retrieval: irrelevant chunks are returned, so the LLM answers from wrong information.
- No grounding instruction: the LLM may ignore context and hallucinate.
- Outdated data: the vector database is not updated, so answers are stale.

9. One-Line Truth of RAG

RAG uses embeddings to fetch the right knowledge and an LLM to turn that knowledge into a clear, human-readable answer.